

```

GUID that is newly generated
    /// by FD for every request that it makes to the downstream
services.

    /// </summary>
    public const string MSRequestIdHeader = "MS-RequestId";
}

public static class HttpMimeTypes
{
    /// <summary>
    /// The json content type
    /// </summary>
    public const string JsonContentType = "application/json";
}
}
}
}

```

Related topics

- [Create and manage submissions using Microsoft Store services](#)

C# sample: app submission with game options and trailers

Article • 01/04/2023

This article provides C# code examples that demonstrate how to use the [Microsoft Store submission API](#) for these tasks:

- Obtain an Azure AD access token to use with the Microsoft Store submission API.
- Create an app submission
- Configure Store listing data for the app submission, including the [gaming](#) and [trailers](#) advanced listing options.
- Upload the ZIP file containing the packages, listing images, and trailer files for the app submission.
- Commit the app submission.

You can review each example to learn more about the task it demonstrates, or you can build all the code examples in this article into a console application. To build the examples, create a C# console application named **DevCenterApiSample** in Visual Studio, copy each example to a separate code file in the project, and build the project.

Prerequisites

These examples have the following requirements:

- Add a reference to the System.Web assembly in your project.
- Install the [Newtonsoft.Json](#)  NuGet package from Newtonsoft to your project.

Create an app submission

The `CreateAndSubmitSubmissionExample` class defines a public `Execute` method that calls other example methods to use the Microsoft Store submission API to create and commit an app submission that contains game options and a trailer. To adapt this code for your own use:

- Assign the `tenantId` variable to the tenant ID for your app, and assign the `clientId` and `clientSecret` variables to the client ID and key for your app. For more information, see [How to associate an Azure AD application with your Partner Center account](#)

- Assign the `applicationId` variable to the [Store ID](#) of the app for which you want to create a submission.

C#

```
using System;
using System.Threading;
using Newtonsoft.Json.Linq;

namespace DevCenterApiSample
{
    public class CreateAndSubmitSubmissionExample
    {
        public static void Execute()
        {
            // Add your tenant ID, client ID, and client secret here.
            string tenantId = "";
            string clientId = "";
            string clientSecret = "";
            var accessTokenClient = new DevCenterAccessTokenClient(tenantId,
clientId, clientSecret);

            string accessToken =
accessTokenClient.GetAccessToken("https://manage.devcenter.microsoft.com");
            var devCenter = new DevCenterClient(accessToken);

            // The application ID is taken from your app dashboard page's
            // URI in Dev Center,
            // e.g. https://developer.microsoft.com/en-
            // us/dashboard/apps/{application_id}/
            string applicationId = "{application_id}";

            // Get the application object, and cancel any in progress
            // submissions.
            JObject app = devCenter.GetApplication(applicationId);
            JToken inProgressSubmission =
app.GetValue("pendingApplicationSubmission");
            if (inProgressSubmission != null)
            {
                string inProgressSubmissionId =
inProgressSubmission.Value<string>("id");
                devCenter.CancelInProgressSubmission(applicationId,
inProgressSubmissionId);
            }

            // Create a new submission, based on the last published
            // submission.
            JObject submission = devCenter.CreateSubmission(applicationId);
            string submissionId = submission.GetValue("id").Value<string>();

            // The following fields are required.
            submission["applicationCategory"] = "Games_Fighting";
            submission["listings"] = GetListingsObject();
            submission["pricing"] = GetPricingObject();
        }
    }
}
```

```

        submission["packages"] = new JSONArray() { GetPackageObject() };
        submission["allowTargetFutureDeviceFamilies"] =
GetDeviceFamiliesObject();

        // The app must have the hasAdvancedListingPermission set to
True in order for gaming options
        // and trailers to be applied. If that's not the case, you can
still update the app and
        // its submissions through the API, but gaming options and
trailers won't be saved.
        if (app["hasAdvancedListingPermission"] == null ||
app["hasAdvancedListingPermission"].Value<bool>() == false)
        {
            Console.WriteLine("This application does not support gaming
options or trailers.");
        }
        else
        {
            // Gaming options is an array. A maximum of one value may be
provided.
            submission["gamingOptions"] = new
JSONArray(GetGamingOptionsObject());

            // A maximum of 15 trailers may be provided in the trailers
array.
            submission["trailers"] = new JSONArray(GetTrailerObject());
        }

        // Continue updating the submission_json object with additional
options as needed.
        // After you've finished, call the Update API with the code
below to save it.
        JObject updatedSubmission =
devCenter.UpdateSubmission(applicationId, submissionId, submission);

        // All images and packages should be located in a single ZIP
file. In the submission JSON,
        // the file names for all objects requiring them (icons,
packages, etc.) must exactly
        // match the file names from the ZIP file.
        string zipFilePath = "";
        devCenter.UploadZipFileForSubmission(applicationId,
submissionId, zipFilePath);

        // Committing the submission will start the submission process
for it. Once committed,
        // the submission can no longer be changed.
        devCenter.CommitSubmission(applicationId, submissionId);

        // After committing, you can poll the commit API for the status
of the submission's process using
        // the following code.
        bool waitingForCommitToStart = true;
        while (waitingForCommitToStart)
        {

```

```

        string status = devCenter.GetSubmissionStatus(applicationId,
submissionId);

        Console.WriteLine($"Submission status: {status}");
        waitingForCommitToStart = status.Equals("CommitStarted");
        if (waitingForCommitToStart)
        {
            Thread.Sleep(TimeSpan.FromMinutes(1)); // Wait to check
Dev Center again.
        }
    }
}

private static JObject GetListingsObject()
{
    // This structure holds basic information to display in the
store.

    var baseListing = new JObject();
    baseListing.Add("copyrightAndTrademarkInfo", "(C) 2017
Microsoft");
    baseListing.Add("licenseTerms",
"http://example.com/licenseTerms.aspx");
    baseListing.Add("privacyPolicy",
"http://example.com/privacyPolicy.aspx");
    baseListing.Add("supportContact", "support@example.com");
    baseListing.Add("websiteUrl", "http://example.com");
    baseListing.Add("description", "A sample game showing off
gameplay options code.");
    baseListing.Add("releaseNotes", "Initial release");

    // The title of the app must match a reserved name for the app
in Dev Center.
    // If it doesn't, attempting to update the submission will fail.
    baseListing.Add("title", "Super Game Options API Simulator
2017");

    var keywords = new JArray();
    keywords.Add("SampleApp");
    keywords.Add("SampleFightingGame");
    keywords.Add("GameOptions");
    baseListing.Add("keywords", keywords);

    var features = new JArray();
    features.Add("Doesn't crash");
    features.Add("Likes to eat chips");
    baseListing.Add("features", features);

    // If your app works better with specific hardware (or needs
it), you can
    // add or update values here.
    var hardwarePreferences = new JArray()
    {
        "Keyboard",
        "Mouse"
    };
    baseListing.Add("hardwarePreferences", hardwarePreferences);

```

```

        var images = new JSONArray();

        // There are several types of images available; at least one
screenshot
        // is required.
        var image = new JObject();

        // The file name is relative to the root of the uploaded ZIP
file.
        image.Add("fileName", "img/screenshot.png");
        image.Add("description", "A basic screenshot of the app.");
        image.Add("imageType", "Screenshot");
        images.Add(image);
        baseListing.Add("images", images);

        var listing = new JObject();
        listing.Add("baseListing", baseListing);

        // If there are any specific overrides to above information for
Windows 8,
        // Windows 8.1, Windows Phone 7.1, 8.0, or 8.1, you can add
information here.
        listing.Add("platformOverrides", new JObject());

        // Each listing is targeted at a specific language-locale code,
e.g. EN-US.
        var listings = new JObject();
        listings.Add("en-us", listing);
        return listings;
    }

    private static JObject GetPackageObject()
    {
        var package = new JObject()
        {
            // The file name is relative to the root of the uploaded ZIP
file.
            ["fileName"] = "bin/super_dev_ctr_api_sim.appxupload",

            // If you haven't begun to upload the file yet, set this
value to "PendingUpload".
            ["fileStatus"] = "PendingUpload"
        };
        return package;
    }

    private static JObject GetPricingObject()
    {
        var pricing = new JObject();

        // How long the trial period is, if one is allowed. Valid values
are NoFreeTrial,
        // OneDay, SevenDays, FifteenDays, ThirtyDays, or
TrialNeverExpires.

```

```

        pricing.Add("trialPeriod", "NoFreeTrial");

        // Maps to the default price for the app.
        pricing.Add("priceId", "Free");

        // If you'd like to offer your app in different markets at
different prices, you
        // can provide priceId values per language/locale code.
        pricing.Add("marketSpecificPricing", new JObject());
        return pricing;
    }

    private static JObject GetDeviceFamiliesObject()
    {
        var futureDeviceFamilies = new JObject();

        // Supported values are Desktop, Mobile, Xbox, and Holographic.
        // the app available on that specific platform, set the value to
To make
True.

        futureDeviceFamilies.Add("Desktop", true);
        futureDeviceFamilies.Add("Mobile", false);
        futureDeviceFamilies.Add("Xbox", true);
        futureDeviceFamilies.Add("Holographic", false);
        return futureDeviceFamilies;
    }

    private static JObject GetTrailerObject()
    {
        // Add an example trailer.
        var trailer = new JObject();

        // This is the filename of the trailer. The file name is a
relative path to the
        // root of the ZIP file to be uploaded to the API.
        trailer["VideoFileName"] =
"trailers/main/my_awesome_trailer.mpeg";

        // Aside from the video itself, a trailer can have image assets
such as screenshots
        // or alternate images.
        var trailerAssets = new JObject();
        trailer["TrailerAssets"] = trailerAssets;

        // Add trailer assets for the EN-US market.
        var trailerAsset = new JObject();
        trailerAssets["en-us"] = trailerAsset;

        // The title of the trailer to display in the store.
        trailerAsset["Title"] = "Main Trailer";

        // The list of images provided with the trailer that are shown
        // when the trailer isn't playing.
        var imageUrl = new JArray();
        trailerAsset["ImageList"] = imageUrl;

```

```

// Add a few images to the image list.
var thumbnailImage = new JObject()
{
    // The file name of the image. The file name is a relative
    // path to the root of the ZIP
    // file to be uploaded to the API.
    ["FileName"] = "trailers/main/thumbnail.png",

    // A plaintext description of what the image represents.
    ["Description"] = "The thumbnail for the trailer shown " +
"before the user clicks play"
};
imageList.Add(thumbnailImage);

var altImage = new JObject()
{
    ["FileName"] = "trailers/main/alt-img.png",
    ["Description"] = "The image to show after the trailer
plays"
};
imageList.Add(altImage);

return trailer;
}

private static JObject GetGamingOptionsObject()
{
    var gamingOptions = new JObject();

    // The genres of your game.
    var genres = new JArray();
    genres.Add("Games_Fighting");
    gamingOptions["genres"] = genres;

    // Set this to true if your game supports local multiplayer.
This field
    // is required.
    gamingOptions["isLocalMultiplayer"] = true;

    // If local multiplayer is supported, you must provide the
minimum and
    // maximum players supported. Valid values are between 2 and
1000 inclusive.
    gamingOptions["localMultiplayerMinPlayers"] = 2;
    gamingOptions["localMultiplayerMaxPlayers"] = 4;

    // Set this to True if your game supports local co-op play. This
field is required.
    gamingOptions["isLocalCooperative"] = true;

    // If local co-op is supported, you must provide the minimum and
maximum players
    // supported. Valid values are between 2 and 1000 inclusive.
    gamingOptions["localCooperativeMinPlayers"] = 2;

```



```

        gamingOptions["localCooperativeMaxPlayers"] = 4;

        // Set this to True if your game supports online multiplayer.
        This field is required.
        gamingOptions["isOnlineMultiplayer"] = true;

        // If online multiplayer is supported, you must provide the
        minimum and maximum players
        // supported. Valid values are between 2 and 1000 inclusive.
        gamingOptions["onlineMultiplayerMinPlayers"] = 2;
        gamingOptions["onlineMultiplayerMaxPlayers"] = 4;

        // Set this to true if your game supports online co-op play.
        This field is required.
        gamingOptions["isOnlineCooperative"] = true;

        // If online co-op is supported, you must provide the minimum
        and maximum players
        // supported. Valid values are between 2 and 1000 inclusive.
        gamingOptions["onlineCooperativeMinPlayers"] = 2;
        gamingOptions["onlineCooperativeMaxPlayers"] = 4;

        // If your game supports broadcasting a stream to other players,
        set this field to True.
        // This field is required.
        gamingOptions["isBroadcastingPrivilegeGranted"] = true;

        // If your game supports cross-device play (e.g. a player can
        play on an Xbox One with
        // their friend who's playing on a PC), set this field to True.
        This field is required.
        gamingOptions["isCrossPlayEnabled"] = true;

        // If your game supports Kinect usage, set this field to
        "Enabled", otherwise, set it to
        // "Disabled". This field is required.
        gamingOptions["kinectDataForExternal"] = "Disabled";

        // Free text about any other peripherals that your game
        supports. This field is optional.
        gamingOptions["otherPeripherals"] = "Supports the usage of all
        fighting joysticks.";
        return gamingOptions;
    }
}
}

```

Obtain an Azure AD access token

The `DevCenterAccessTokenClient` class defines a helper method that uses the your `tenantId`, `clientId` and `clientSecret` values to create an Azure AD access token to use

with the Microsoft Store submission API.

C#

```
using System.Net.Http;
using System.Text;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;

namespace DevCenterApiSample
{
    /// <summary>
    /// A client for getting access tokens to the Dev Center API.
    /// </summary>
    public class DevCenterAccessTokenClient
    {
        private string _tenantId;
        private string _clientId;
        private string _clientSecret;

        /// <summary>
        /// Creates a new instance of the <see
        cref="DevCenterAccessTokenClient"/> class.
        /// </summary>
        /// <param name="tenantId">The AAD tenant ID.</param>
        /// <param name="clientId">The AAD client ID.</param>
        /// <param name="clientSecret">The AAD client secret.</param>
        public DevCenterAccessTokenClient(string tenantId, string clientId,
        string clientSecret)
        {
            _tenantId = tenantId;
            _clientId = clientId;
            _clientSecret = clientSecret;
        }

        /// <summary>
        /// Generates an access token to the specified resource URI.
        /// </summary>
        /// <param name="resource">The resource URI.</param>
        /// <returns>An access token for authentication.</returns>
        public string GetAccessToken(string resource)
        {
            // Generate access token. Access token is valid for 1 hour.
            Regenerate access token when needed
            HttpRequestMessage tokenRequest = new
            HttpRequestMessage(HttpMethod.Post,
            $"https://login.microsoftonline.com/{_tenantId}/oauth2/token");
            string tokenRequestBody =
            $"grant_type=client_credentials&client_id={_clientId}&client_secret=
            {_clientSecret}&resource={resource}";
            tokenRequest.Content = new StringContent(tokenRequestBody,
            Encoding.UTF8, "application/x-www-form-urlencoded");

            HttpClient client = new HttpClient();
```

```

        HttpResponseMessage response =
client.SendAsync(tokenRequest).GetAwaiter().GetResult();
        string responseBody =
response.Content.ReadAsStringAsync().GetAwaiter().GetResult();
        JObject responseJson =
(JObject)JsonConvert.DeserializeObject(responseBody);

        tokenRequest.Dispose();
        client.Dispose();
        response.Dispose();

        return responseJson["access_token"].Value<string>() ??
string.Empty;
    }
}

```

Helper methods to invoke the submission API and upload submission files

The `DevCenterClient` class defines helper methods that invoke a variety of methods in the Microsoft Store submission API and upload the ZIP file containing the packages, listing images, and trailer files for the app submission.

C#

```

using System;
using System.Net.Http;
using System.Net.Http.Headers;
using System.Text;
using System.Web;
using Newtonsoft.Json;
using Newtonsoft.Json.Linq;
using System.IO;

namespace DevCenterApiSample
{
    /// <summary>
    /// A client for accessing the Dev Center APIs.
    /// </summary>
    public class DevCenterClient
    {
        private string _accessToken;
        private Uri _baseUri;

        /// <summary>
        /// Creates a new instance of the <see cref="DevCenterClient"/>
class.
        /// </summary>
        /// <param name="accessToken">The access token to authenticate to

```

```

the service with.</param>
    public DevCenterClient(string accessToken)
    {
        _baseUri = new Uri("https://manage.devcenter.microsoft.com");
        _accessToken = accessToken;
    }

    /// <summary>
    /// Retrieves the JSON object representing the application from Dev
Center.
    /// </summary>
    /// <param name="applicationId">The application ID.</param>
    /// <returns>A JObject that may be navigated.</returns>
    /// <remarks>
    /// The application ID is taken from your app dashboard page's URI
in Dev Center,
    /// e.g. https://developer.microsoft.com/en-
us/dashboard/apps/{application_id}/
    /// </remarks>
    public JObject GetApplication(string applicationId)
        => Invoke(HttpMethod.Get,
    $"/v1.0/my/applications/{applicationId}");

    /// <summary>
    /// Cancels an in-progress submission for the app.
    /// </summary>
    /// <param name="applicationId">The application ID.</param>
    /// <param name="submissionId">The submission ID.</param>
    /// <returns></returns>
    public void CancelInProgressSubmission(string applicationId, string
submissionId)
        => Invoke(HttpMethod.Delete,
    $"/v1.0/my/applications/{applicationId}/submissions/{submissionId}");

    /// <summary>
    /// Creates a new in-progress submission for the application.
    /// </summary>
    /// <param name="applicationId">The application ID.</param>
    /// <returns>A JObject that may navigated.</returns>
    public JObject CreateSubmission(string applicationId)
        => Invoke(HttpMethod.Post,
    $"/v1.0/my/applications/{applicationId}/submissions");

    /// <summary>
    /// Updates the submission with the new data provided.
    /// </summary>
    /// <param name="applicationId">The application ID.</param>
    /// <param name="submissionId">The submission ID.</param>
    /// <param name="submission">The submission body.</param>
    /// <returns>The updated submission JObject.</returns>
    public JObject UpdateSubmission(string applicationId, string
submissionId, JObject submission)
        => Invoke(HttpMethod.Put,
    $"/v1.0/my/applications/{applicationId}/submissions/{submissionId}",
    submission);

```

```

    /// <summary>
    /// Gets the submission from Dev Center.
    /// </summary>
    /// <param name="applicationId">The application ID.</param>
    /// <param name="submissionId">The submission ID.</param>
    /// <returns>The submission object from Dev Center.</returns>
    public JObject GetSubmission(string applicationId, string
submissionId)
    {
        => Invoke(HttpMethod.Get,
$"/v1.0/my/applications/{applicationId}/submissions/{submissionId}");

        /// <summary>
        /// Commits the submission to Dev Center.
        /// </summary>
        /// <param name="applicationId">The application ID.</param>
        /// <param name="submissionId">The submission ID.</param>
        /// <remarks>
        /// Once a submission is committed, Dev Center will begin processing
and certifying it;
        /// it can no longer be changed after this point.
        /// </remarks>
        public void CommitSubmission(string applicationId, string
submissionId)
        {
            => Invoke(HttpMethod.Post,
$"/v1.0/my/applications/{applicationId}/submissions/{submissionId}/commit");

            /// <summary>
            /// Returns the current submission commit status.
            /// </summary>
            /// <param name="applicationId">The application ID.</param>
            /// <param name="submissionId">The submission ID.</param>
            /// <returns>The submission status.</returns>
            public string GetSubmissionStatus(string applicationId, string
submissionId)
            {
                JObject response = GetSubmission(applicationId, submissionId);
                string status = response.Value<string>("status") ?? "Unknown";
                return status;
            }

            /// <summary>
            /// Uploads the ZIP file containing assets for the submission to the
submission in Dev Center.
            /// </summary>
            /// <param name="applicationId">The application ID.</param>
            /// <param name="submissionId">The submission ID.</param>
            /// <param name="zipFilePath">The path to the ZIP file.</param>
            public void UploadZipFileForSubmission(string applicationId, string
submissionId, string zipFilePath)
            {
                JObject submission = GetSubmission(applicationId, submissionId);
                string fileUploadUrl = submission["fileUploadUri"].Value<string>
());

```

```

        HttpRequestMessage uploadRequest = new
HttpRequestMessage(HttpMethod.Put, fileUploadUrl.Replace("+", "%2B")); //
Encode '+', otherwise it will be decoded as ' '
        uploadRequest.Content = new
StreamContent(File.OpenRead(zipFilePath));
        uploadRequest.Headers.Add("x-ms-blob-type", "BlockBlob");

        HttpClient httpClient = new HttpClient();
        HttpResponseMessage uploadResponse =
httpClient.SendAsync(uploadRequest).GetAwaiter().GetResult();
        uploadResponse.EnsureSuccessStatusCode();

        uploadRequest.Dispose();
        uploadResponse.Dispose();
        httpClient.Dispose();
    }

    private JObject Invoke(HttpMethod method, string path, JObject body
= null)
    {
        HttpRequestMessage request = new HttpRequestMessage(method, new
Uri(_baseUri, path));
        request.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", _accessToken);
        request.Headers.UserAgent.ParseAdd("C-Sharp");
        if (body != null)
        {
            request.Content = new StringContent(body.ToString(),
Encoding.UTF8, "application/json");
        }

        HttpClient client = new HttpClient();
        HttpResponseMessage response =
client.SendAsync(request).GetAwaiter().GetResult();
        string responseContent =
response.Content.ReadAsStringAsync().GetAwaiter().GetResult();

        if (!response.IsSuccessStatusCode)
        {
            string message = string.IsNullOrEmpty(responseContent) ?
response.ReasonPhrase : responseContent;
            throw new HttpException((int)response.StatusCode, message);
        }

        if (string.IsNullOrEmpty(responseContent))
        {
            return null;
        }

        client.Dispose();
        request.Dispose();
        response.Dispose();

        JObject responseObject =
(JObject)JsonConvert.DeserializeObject(responseContent);
    }

```

```
        return responseObject;  
    }  
}
```

Related topics

- [Create and manage submissions using Microsoft Store services](#)

Java sample: submissions for apps, add-ons, and flights

Article • 01/04/2023

This article provides Java code examples that demonstrate how to use the [Microsoft Store submission API](#) for these tasks:

- [Obtain an Azure AD access token](#)
- [Create an add-on](#)
- [Create a package flight](#)
- [Create an app submission](#)
- [Create an add-on submission](#)
- [Create a package flight submission](#)

You can review each example to learn more about the task it demonstrates, or you can build all the code examples in this article into a console application. For the complete code listing, see the [code listing](#) section at the end of this article.

Prerequisites

These examples use the following libraries:

- [Apache Commons Logging 1.2](#) [↗](#) (commons-logging-1.2.jar).
- [Apache HttpComponents Core 4.4.5](#) and [Apache HttpComponents Client 4.5.2](#) [↗](#) (httpcore-4.4.5.jar and httpclient-4.5.2.jar).
- [JSR 353 JSON Processing API 1.0](#) [↗](#) and [JSR 353 JSON Processing Default Provider API 1.0.4](#) [↗](#) (javax.json-api-1.0.jar and javax.json-1.0.4.jar).

Main program and imports

The following example shows the imports statements used by all of the code examples and implements a command line program that calls the other example methods.

Java

```
import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.IOException;
import java.io.UnsupportedEncodingException;
import java.net.URISyntaxException;
import java.text.MessageFormat;
```



```

import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.StatusLine;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.HttpResponseException;
import org.apache.http.client.ResponseHandler;
import org.apache.http.client.methods.*;
import org.apache.http.impl.client.BasicResponseHandler;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.entity.FileEntity;
import org.apache.http.entity.StringEntity;

import javax.json.Json;
import javax.json.JsonObject;
import javax.json.JsonReader;

public class IngestionServiceJavaSamples {

    public static void main(String[] args) throws InterruptedException,
        URISyntaxException, IOException {

        // Generate access token. Access token is valid for 1 hour.
        Regenerate access token when needed
        String tenantId = ""; // {Your tenant ID}
        String clientId = ""; // {Your client ID}
        String clientSecret = ""; // {Your client secret}

        String accessToken = GenerateAccessToken(tenantId, clientId,
            clientSecret);

        // Application submission sample
        String applicationId = ""; // {Your application ID}
        String appSubmissionRequestJson = ""; // {Your submission request
JSON}
        String appSubmissionZipFilePath = ""; // {Your zip file path}

        SubmitNewApplicationSubmission(accessToken, applicationId,
            appSubmissionRequestJson,
            appSubmissionZipFilePath);

        // Flight samples
        String flightId = ""; // {Your flight ID}
        String flightRequestJson = ""; // {Your flight request
JSON}
        String flightSubmissionRequestJson = ""; // {Your submission
request JSON}
        String flightSubmissionZipFilePath = ""; // {Your zip file path}

        CreateNewFlight(accessToken, applicationId, flightRequestJson);
        SubmitNewFlightSubmission(accessToken, applicationId, flightId,
            flightSubmissionRequestJson,
            flightSubmissionZipFilePath);

        // In-app-product samples

```

```

        String inAppProductId = "";           //{Your in-app-product ID}
        String inAppProductRequestJson = "";   //{Your in-app-product
request JSON}
        String iapSubmissionRequestJson = "";  //{Your submission request
JSON}
        String iapSubmissionZipFilePath = "";  //{Your zip file path}

        CreateNewInAppProduct(accessToken, inAppProductRequestJson);
        SubmitNewInAppProductSubmission(accessToken, inAppProductId,
iapSubmissionRequestJson,
            iapSubmissionZipFilePath);
    }
}

```

Obtain an Azure AD access token

The following example demonstrates how to [obtain an Azure AD access token](#) that you can use to call methods in the Microsoft Store submission API. After you obtain a token, you have 60 minutes to use this token in calls to the Microsoft Store submission API before the token expires. After the token expires, you can generate a new token.

Java

```

public static String GenerateAccessToken(String tenantId, String clientId,
String clientSecret) {

    // Generate access token. Access token is valid for 1 hour. Regenerate
access token when needed
    String tokenEndpoint =
"https://login.microsoftonline.com/{0}/oauth2/token";
    String tokenResource = "https://manage.devcenter.microsoft.com";

    HttpPost tokenRequest = new HttpPost(MessageFormat.format(tokenEndpoint,
tenantId));
    String tokenRequestBody = MessageFormat.format(
        "grant_type=client_credentials&client_id={0}&client_secret=
{1}&resource={2}",
        clientId, clientSecret, tokenResource);
    tokenRequest.setEntity(new StringEntity(tokenRequestBody, "utf-8"));
    tokenRequest.addHeader("Content-Type", "application/x-www-form-
urlencoded; charset=utf-8");

    CloseableHttpClient httpclient = HttpClients.createDefault();
    ResponseHandler<String> responseHandler = new BasicResponseHandler();

    try {
        String tokenResponse = httpclient.execute(tokenRequest,
responseHandler);
        JsonReader reader = Json.createReader(new
ByteArrayInputStream(tokenResponse.getBytes("UTF-8")));
    }
}

```

```

        String accessToken = reader.readObject().getString("access_token");
        return accessToken;
    } catch (UnsupportedEncodingException e) {
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }

    return null;
}

```

Create an add-on

The following example demonstrates how to [create](#) and then [delete](#) an add-on.

Java

```

public static void CreateNewInAppProduct(String accessToken, String
inAppProductRequestJson)
    throws InterruptedException, IOException {

    CloseableHttpClient httpClient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Create a new in-app-product
        HttpPost createInAppProductRequest = new HttpPost(
            "https://manage.devcenter.microsoft.com/v1.0/my/inappproducts");
        createInAppProductRequest.addHeader("Authorization", "Bearer " +
accessToken);
        createInAppProductRequest.addHeader("Content-type",
"application/json");
        createInAppProductRequest.addHeader("User-Agent", "Java");
        createInAppProductRequest.setEntity(new
StringEntity(inAppProductRequestJson));
        JsonObject createInAppProductResponse =
httpClient.execute(createInAppProductRequest,
responseHandler);
        String inAppProductId = createInAppProductResponse.getString("id");
        System.out.println(createInAppProductResponse.toString());

        // Delete created in-app-product
        HttpDelete deleteInAppProductRequest = new HttpDelete(
            "https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/"
+ inAppProductId);
        deleteInAppProductRequest.addHeader("Authorization", "Bearer " +
accessToken);
        deleteInAppProductRequest.addHeader("User-Agent", "Java");
        httpClient.execute(deleteInAppProductRequest, responseHandler);
    }
}

```

```

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpclient.close();
    }
}

```

Create a package flight

The following example demonstrates how to [create](#) and then [delete](#) a package flight.

Java

```

public static void CreateNewFlight(String accessToken, String applicationId,
String flightRequestJson)
    throws InterruptedException, IOException {

    CloseableHttpClient httpclient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Create a new flight, a flight submission will be created together
with the flight
        HttpPost createFlightRequest = new HttpPost(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights",
applicationId));
        createFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
        createFlightRequest.addHeader("Content-type", "application/json");
        createFlightRequest.setEntity(new StringEntity(flightRequestJson));
        JsonObject createFlightResponse =
httpclient.execute(createFlightRequest, responseHandler);

        System.out.println(createFlightResponse.toString());

        String flightId = createFlightResponse.getString("flightId");
        String submissionId =
createFlightResponse.getJSONObject("pendingFlightSubmission").getString("id"
);

        // Delete created flight
        HttpDelete deleteFlightRequest = new
HttpDelete(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}

```

```

",
        applicationId, flightId));
deleteFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
httpClient.execute(deleteFlightRequest, responseHandler);

} catch (HttpResponseException e){
    e.printStackTrace();
} catch (ClientProtocolException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    httpClient.close();
}
}

```

Create an app submission

The following example shows how to use several methods in the Microsoft Store submission API to create an app submission. To do this, the `SubmitNewApplicationSubmission` method creates a new submission as a clone of the last published submission, and then it updates and commits the cloned submission to Partner Center. Specifically, the `SubmitNewApplicationSubmission` method performs these tasks:

1. To begin, the method [gets data for the specified app](#).
2. Next, it [deletes the pending submission for the app](#), if one exists.
3. It then [creates a new submission for the app](#) (the new submission is a copy of the last published submission).
4. It changes some details for the new submission and upload a new package for the submission to Azure Blob Storage.
5. Next, it [updates](#) and then [commits](#) the new submission to Partner Center.
6. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

Java

```

public static void SubmitNewApplicationSubmission(String accessToken, String
applicationId,
    String appSubmissionRequestJson, String zipFilePath)
    throws InterruptedException, IOException {

    CloseableHttpClient httpClient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

```

```

try {
    // Get application
    HttpGet getApplicationRequest = new HttpGet(
        "https://manage.devcenter.microsoft.com/v1.0/my/applications/" +
applicationId);
    getApplicationRequest.addHeader("Authorization", "Bearer " +
accessToken);
    getApplicationRequest.addHeader("User-Agent", "Java");
    JsonObject getApplicationResponse =
httpClient.execute(getApplicationRequest, responseHandler);

    // Delete existing in-progress submission
    JsonObject submissionToRemove =
getApplicationResponse.getJsonObject(
        "pendingApplicationSubmission");
    if(submissionToRemove != null){
        String submissionToRemoveId =
submissionToRemove.getString("id");
        HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
            applicationId, submissionToRemoveId));
        deleteSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        deleteSubmissionRequest.addHeader("User-Agent", "Java");
        httpClient.execute(deleteSubmissionRequest, responseHandler);
    }

    // Create submission
    HttpPost createSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
",
        applicationId));
        createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        createSubmissionRequest.addHeader("User-Agent", "Java");
        JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);

        String submissionId = createSubmissionResponse.getString("id");
        String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

    // Update submission
    HttpPut updateSubmissionRequest = new HttpPut(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
        applicationId, submissionId));
        updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);

```

```

        updateSubmissionRequest.addHeader("Content-type",
"application/json");
        updateSubmissionRequest.addHeader("User-Agent", "Java");
        updateSubmissionRequest.setEntity(new
StringEntity(appSubmissionRequestJson));
        JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

        // Upload images and packages in a zip file
        UploadZipFile(fileUploadUrl, zipFilePath);

        // Commit submission
        HttpPost commitSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}/commit",
        applicationId, submissionId));
        commitSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        commitSubmissionRequest.addHeader("User-Agent", "Java");
        JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);
        System.out.println(commitSubmissionResponse.getString("status"));

        // Pull submission status until commit process is completed
        HttpGet getSubmissionStatusRequest = new
HttpGet(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
        applicationId, submissionId));
        getSubmissionStatusRequest.addHeader("Authorization", "Bearer " +
accessToken);
        getSubmissionStatusRequest.addHeader("User-Agent", "Java");
        JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
        responseHandler);
        String submissionStatus =
getSubmissionStatusResponse.getString("status");
        while(submissionStatus.equals("CommitStarted")){
            Thread.sleep(60*1000);
            getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
            submissionStatus =
getSubmissionStatusResponse.getString("status");
        }
        System.out.println(submissionStatus);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }

```

```
    } finally {  
        httpClient.close();  
    }  
}
```

Create an add-on submission

The following example shows how to use several methods in the Microsoft Store submission API to create an add-on submission. To do this, the

`SubmitNewInAppProductSubmission` method creates a new submission as a clone of the last published submission, and then updates and commits the cloned submission to Partner Center. Specifically, the `SubmitNewInAppProductSubmission` method performs these tasks:

1. To begin, the method [gets data for the specified add-on](#).
2. Next, it [deletes the pending submission for the add-on](#), if one exists.
3. It then [creates a new submission for the add-on](#) (the new submission is a copy of the last published submission).
4. It uploads a ZIP archive that contains icons for the submission to Azure Blob Storage.
5. Next, it [updates](#) and then [commits](#) the new submission to Partner Center.
6. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

Java

```
public static void SubmitNewInAppProductSubmission(String accessToken,  
String inAppProductId,  
String iapSubmissionRequestJson, String zipFilePath)  
    throws InterruptedException, IOException {  
  
    CloseableHttpClient httpClient = HttpClients.createDefault();  
    ResponseHandler<JsonObject> responseHandler =  
        CreateJsonResponseHandler();  
  
    try {  
        // Get the in-app-product  
        HttpGet getInAppProductRequest = new HttpGet(  
            "https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/"  
            + inAppProductId);  
        getInAppProductRequest.addHeader("Authorization", "Bearer " +  
            accessToken);  
        getInAppProductRequest.addHeader("User-Agent", "Java");  
        JsonObject getInAppProductResponse =  
            httpClient.execute(getInAppProductRequest, responseHandler);  
  
        // Delete existing in-progress submission
```



```

        JsonObject submissionToRemove =
getInAppProductResponse.getJsonObject(
    "pendingInAppProductSubmission");
    if(submissionToRemove != null){
        String submissionToRemoveId =
submissionToRemove.getString("id");
        HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}",

            inAppProductId, submissionToRemoveId));
        deleteSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        deleteSubmissionRequest.addHeader("User-Agent", "Java");
        httpClient.execute(deleteSubmissionRequest, responseHandler);
    }

    // Create submission
    HttpPost createSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s",

            inAppProductId));
        createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        createSubmissionRequest.addHeader("User-Agent", "Java");
        JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);
        String submissionId = createSubmissionResponse.getString("id");
        String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

        // Update submission
        HttpPut updateSubmissionRequest = new HttpPut(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}",

            inAppProductId, submissionId));
        updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        updateSubmissionRequest.addHeader("Content-type",
"application/json");
        updateSubmissionRequest.setEntity(new
StringEntity(iapSubmissionRequestJson));
        JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

        // Upload images and packages in a zip file
        UploadZipFile(fileUploadUrl, zipFilePath);

        // Commit submission
        HttpPost commitSubmissionRequest = new
HttpPost(MessageFormat.format(

```

```

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submissions/{1}/commit",
    inAppProductId, submissionId));
commitSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
commitSubmissionRequest.addHeader("User-Agent", "Java");
JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);
System.out.println(commitSubmissionResponse.getString("status"));

// Pull submission status until commit process is completed
HttpGet getSubmissionStatusRequest = new
HttpGet(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submissions/{1}/status",
    inAppProductId, submissionId));
getSubmissionStatusRequest.addHeader("Authorization", "Bearer " +
accessToken);
getSubmissionStatusRequest.addHeader("User-Agent", "Java");
JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
    responseHandler);
String submissionStatus =
getSubmissionStatusResponse.getString("status");
while(submissionStatus.equals("CommitStarted")){
    Thread.sleep(60*1000);
    getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
    submissionStatus =
getSubmissionStatusResponse.getString("status");
}
System.out.println(submissionStatus);

} catch (HttpResponseException e){
    e.printStackTrace();
} catch (ClientProtocolException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    httpClient.close();
}
}

```

Create a package flight submission

The following example shows how to use several methods in the Microsoft Store submission API to create a package flight submission. To do this, the

`SubmitNewFlightSubmission` method creates a new submission as a clone of the last

published submission, and then updates and commits the cloned submission to Partner Center. Specifically, the `SubmitNewFlightSubmission` method performs these tasks:

1. To begin, the method [gets data for the specified package flight](#).
2. Next, it [deletes the pending submission for the package flight](#), if one exists.
3. It then [creates a new submission for the package flight](#) (the new submission is a copy of the last published submission).
4. It uploads a new package for the submission to Azure Blob Storage.
5. Next, it [updates](#) and then [commits](#) the new submission to PartnerCenter.
6. Finally, it periodically [checks the status of the new submission](#) until the submission is successfully committed.

Java

```
public static void SubmitNewFlightSubmission(String accessToken, String
applicationId,
    String flightId, String flightSubmissionRequestJson, String zipFilePath)
    throws InterruptedException, URISyntaxException, IOException {

    CloseableHttpClient httpClient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Get flight
        HttpGet getFlightRequest = new HttpGet(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
", applicationId,
        flightId));
        getFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
        getFlightRequest.addHeader("User-Agent", "Java");
        JsonObject getFlightResponse = httpClient.execute(getFlightRequest,
responseHandler);

        // Delete existing in-progress submission
        JsonObject submissionToRemove =
getFlightResponse.getJsonObject("pendingFlightSubmission");
        if(submissionToRemove != null){
            String submissionToRemoveId =
submissionToRemove.getString("id");
            HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
        applicationId, flightId, submissionToRemoveId));
            deleteSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
            deleteSubmissionRequest.addHeader("User-Agent", "Java");
```

```

        httpClient.execute(deleteSubmissionRequest, responseHandler);
    }

    // Create submission
    HttpPost createSubmissionRequest = new
    HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions",
        applicationId, flightId));
    createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
    createSubmissionRequest.addHeader("User-Agent", "Java");
    JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);
    String submissionId = createSubmissionResponse.getString("id");
    String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

    // Update submission
    HttpPut updateSubmissionRequest = new HttpPut(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
        applicationId, flightId, submissionId));
    updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
    updateSubmissionRequest.addHeader("Content-type",
"application/json");
    updateSubmissionRequest.addHeader("User-Agent", "Java");
    updateSubmissionRequest.setEntity(new
StringEntity(flightSubmissionRequestJson));
    JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

    // Upload images and packages in a zip file
    UploadZipFile(fileUploadUrl, zipFilePath);

    // Commit submission
    HttpPost commitSubmissionRequest = new
    HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}/commit",
        applicationId, flightId, submissionId));
    commitSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
    commitSubmissionRequest.addHeader("User-Agent", "Java");
    JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);
    System.out.println(commitSubmissionResponse.getString("status"));

    // Pull submission status until commit process is completed
    HttpGet getSubmissionStatusRequest = new
    HttpGet(MessageFormat.format(

```

```

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
    applicationId, flightId, submissionId));
getSubmissionStatusRequest.addHeader("Authorization", "Bearer " +
accessToken);
getSubmissionStatusRequest.addHeader("User-Agent", "Java");
JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
    responseHandler);
String submissionStatus =
getSubmissionStatusResponse.getString("status");
while(submissionStatus.equals("CommitStarted")){
    Thread.sleep(60*1000);
    getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
    submissionStatus =
getSubmissionStatusResponse.getString("status");
}
System.out.println(submissionStatus);

} catch (HttpResponseException e){
    e.printStackTrace();
} catch (ClientProtocolException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    httpClient.close();
}
}
}

```

Utility methods to upload submission files and handle request responses

The following utility methods demonstrate these tasks:

- How to upload a ZIP archive containing new assets for an app or add-on submission to Azure Blob Storage. For more information about uploading a ZIP archive to Azure Blob Storage for app and add-on submissions, see the relevant instructions in [Create an app submission](#), [Create an add-on submission](#), and [Create a package flight submission](#).
- How to handle request responses.

Java

```

private static void UploadZipFile(String fileUploadUrl, String zipFilePath)
throws IOException{
    CloseableHttpClient httpClient = HttpClients.createDefault();

```

```

File uploadFile = new File(zipFilePath);

// Encode '+', otherwise it will be decoded as ' '
HttpPut uploadFileRequest = new HttpPut(fileUploadUrl.replace("+",
"%2B"));
uploadFileRequest.addHeader("x-ms-blob-type", "BlockBlob");
uploadFileRequest.setEntity(new FileEntity(uploadFile));

CloseableHttpResponse uploadFileResponse;
try {
    uploadFileResponse = httpClient.execute(uploadFileRequest);
    System.out.println(uploadFileResponse.getStatusLine());
} catch (ClientProtocolException e) {
    e.printStackTrace();
} catch (IOException e) {
    e.printStackTrace();
} finally {
    httpClient.close();
}
}

private static ResponseHandler<JsonObject> CreateJsonResponseHandler(){
    ResponseHandler<JsonObject> responseHandler = new
ResponseHandler<JsonObject>() {
        @Override
        public JsonObject handleResponse(HttpResponse response) throws
IOException {
            StatusLine statusLine = response.getStatusLine();
            HttpEntity entity = response.getEntity();
            JsonObject returnValue = null;
            if(entity != null){
                JsonReader reader = Json.createReader(entity.getContent());
                returnValue = reader.readObject();
            }

            // Log response status and correlation ID
            System.out.println(statusLine);
            System.out.println(response.getFirstHeader("MS-CorrelationId"));

            // Underlying failure reason is in response entity
            if (statusLine.getStatusCode() >= 300) {
                if(entity == null){
                    throw new HttpResponseException(
                        statusLine.getStatusCode(),
                        statusLine.getReasonPhrase());
                }
                else{
                    throw new HttpResponseException(
                        statusLine.getStatusCode(),
                        statusLine.getReasonPhrase() + " " +
returnValue.toString() );
                }
            }

            return returnValue;
        }
    };
}

```

```

    }
};

return responseHandler;
}

```

Complete code listing

The following code listing contains all of the previous examples organized into one source file.

Java

```

import java.io.ByteArrayInputStream;
import java.io.File;
import java.io.IOException;
import java.io.UnsupportedEncodingException;
import java.net.URISyntaxException;
import java.text.MessageFormat;

import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.StatusLine;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.HttpResponseException;
import org.apache.http.client.ResponseHandler;
import org.apache.http.client.methods.*;
import org.apache.http.impl.client.BasicResponseHandler;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;
import org.apache.http.entity.FileEntity;
import org.apache.http.entity.StringEntity;

import javax.json.Json;
import javax.json.JsonObject;
import javax.json.JsonReader;

public class IngestionServiceJavaSamples {

    public static void main(String[] args) throws InterruptedException,
        URISyntaxException, IOException {

        // Generate access token. Access token is valid for 1 hour.
        // Regenerate access token when needed
        String tenantId = ""; // {Your tenant ID}
        String clientId = ""; // {Your client ID}
        String clientSecret = ""; // {Your client secret}

        String accessToken = GenerateAccessToken(tenantId, clientId,
            clientSecret);
    }
}

```

```

        // Application submission sample
        String applicationId = "";           //{Your application ID}
        String appSubmissionRequestJson = ""; //{Your submission request
JSON}

        String appSubmissionZipFilePath = ""; //{Your zip file path}

        SubmitNewApplicationSubmission(accessToken, applicationId,
appSubmissionRequestJson,
        appSubmissionZipFilePath);

        // Flight samples
        String flightId = "";               //{Your flight ID}
        String flightRequestJson = "";      //{Your flight request
JSON}

        String flightSubmissionRequestJson = ""; //{Your submission
request JSON}
        String flightSubmissionZipFilePath = ""; //{Your zip file path}

        CreateNewFlight(accessToken, applicationId, flightRequestJson);
        SubmitNewFlightSubmission(accessToken, applicationId, flightId,
flightSubmissionRequestJson,
        flightSubmissionZipFilePath);

        // In-app-product samples
        String inAppProductId = "";         //{Your in-app-product ID}
        String inAppProductRequestJson = ""; //{Your in-app-product
request JSON}

        String iapSubmissionRequestJson = ""; //{Your submission request
JSON}

        String iapSubmissionZipFilePath = ""; //{Your zip file path}

        CreateNewInAppProduct(accessToken, inAppProductRequestJson);
        SubmitNewInAppProductSubmission(accessToken, inAppProductId,
iapSubmissionRequestJson,
        iapSubmissionZipFilePath);
    }

    public static String GenerateAccessToken(String tenantId, String
clientId, String clientSecret) {

        // Generate access token. Access token is valid for 1 hour.
        // Regenerate access token when needed
        String tokenEndpoint =
"https://login.microsoftonline.com/{0}/oauth2/token";
        String tokenResource = "https://manage.devcenter.microsoft.com";

        HttpPost tokenRequest = new
HttpPost(MessageFormat.format(tokenEndpoint, tenantId));
        String tokenRequestBody = MessageFormat.format(
            "grant_type=client_credentials&client_id={0}&client_secret=
{1}&resource={2}",
            clientId, clientSecret, tokenResource);
        tokenRequest.setEntity(new StringEntity(tokenRequestBody, "utf-8"));
        tokenRequest.addHeader("Content-Type", "application/x-www-form-
urlencoded; charset=utf-8");
    }

```



```

        CloseableHttpClient httpClient = HttpClients.createDefault();
        ResponseHandler<String> responseHandler = new
BasicResponseHandler();

        try {
            String tokenResponse = httpClient.execute(tokenRequest,
responseHandler);
            JsonReader reader = Json.createReader(new
ByteArrayInputStream(tokenResponse.getBytes("UTF-8")));
            String accessToken =
reader.readObject().getString("access_token");
            return accessToken;
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        } catch (ClientProtocolException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        return null;
    }

    public static void SubmitNewApplicationSubmission(String accessToken,
String applicationId,
String appSubmissionRequestJson, String zipFilePath)
throws InterruptedException, IOException {

        CloseableHttpClient httpClient = HttpClients.createDefault();
        ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

        try {
            // Get application
            HttpGet getApplicationRequest = new HttpGet(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/" +
applicationId);
            getApplicationRequest.addHeader("Authorization", "Bearer " +
accessToken);
            getApplicationRequest.addHeader("User-Agent", "Java");
            JsonObject getApplicationResponse =
httpClient.execute(getApplicationRequest, responseHandler);

            // Delete existing in-progress submission
            JsonObject submissionToRemove =
getApplicationResponse.getJsonObject(
                "pendingApplicationSubmission");
            if(submissionToRemove != null){
                String submissionToRemoveId =
submissionToRemove.getString("id");
                HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(

```

```

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
        applicationId, submissionToRemoveId));
deleteSubmissionRequest.addHeader("Authorization", "Bearer "
+ accessToken);
deleteSubmissionRequest.addHeader("User-Agent", "Java");
httpClient.execute(deleteSubmissionRequest,
responseHandler);
    }

    // Create submission
    HttpPost createSubmissionRequest = new
HttpPost(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
",
        applicationId));
createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
createSubmissionRequest.addHeader("User-Agent", "Java");
JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);

    String submissionId = createSubmissionResponse.getString("id");
    String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

    // Update submission
    HttpPut updateSubmissionRequest = new
HttpPut(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
        applicationId, submissionId));
updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
updateSubmissionRequest.addHeader("Content-type",
"application/json");
updateSubmissionRequest.addHeader("User-Agent", "Java");
updateSubmissionRequest.setEntity(new
StringEntity(appSubmissionRequestJson));
JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

    // Upload images and packages in a zip file
    UploadZipFile(fileUploadUrl, zipFilePath);

    // Commit submission
    HttpPost commitSubmissionRequest = new
HttpPost(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}/commit",
        applicationId, submissionId));
commitSubmissionRequest.addHeader("Authorization", "Bearer " +

```

```

accessToken);
    commitSubmissionRequest.addHeader("User-Agent", "Java");
    JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);

System.out.println(commitSubmissionResponse.getString("status"));

    // Pull submission status until commit process is completed
    HttpGet getSubmissionStatusRequest = new
HttpGet(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/submissions
/{1}",
        applicationId, submissionId));
    getSubmissionStatusRequest.addHeader("Authorization", "Bearer "
+ accessToken);
    getSubmissionStatusRequest.addHeader("User-Agent", "Java");
    JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
        responseHandler);
    String submissionStatus =
getSubmissionStatusResponse.getString("status");
    while(submissionStatus.equals("CommitStarted")){
        Thread.sleep(60*1000);
        getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
        submissionStatus =
getSubmissionStatusResponse.getString("status");
    }
    System.out.println(submissionStatus);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpClient.close();
    }
}

    public static void CreateNewFlight(String accessToken, String
applicationId, String flightRequestJson)
        throws InterruptedException, IOException {

        CloseableHttpClient httpClient = HttpClients.createDefault();
        ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

        try {
            // Create a new flight, a flight submission will be created
            together with the flight
            HttpPost createFlightRequest = new
HttpPost(MessageFormat.format(

```

```

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights",
applicationId));
    createFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
    createFlightRequest.addHeader("Content-type",
"application/json");
    createFlightRequest.setEntity(new
StringEntity(flightRequestJson));
    JsonObject createFlightResponse =
httpClient.execute(createFlightRequest, responseHandler);

    System.out.println(createFlightResponse.toString());

    String flightId = createFlightResponse.getString("flightId");
    String submissionId =
createFlightResponse.getJsonObject("pendingFlightSubmission").getString("id"
);

    // Delete created flight
    HttpDelete deleteFlightRequest = new
HttpDelete(MessageFormat.format(
"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
",
        applicationId, flightId));
    deleteFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
    httpClient.execute(deleteFlightRequest, responseHandler);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpClient.close();
    }
}

public static void SubmitNewFlightSubmission(String accessToken, String
applicationId,
    String flightId, String flightSubmissionRequestJson, String
zipFilePath)
    throws InterruptedException, URISyntaxException, IOException {

    CloseableHttpClient httpClient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Get flight
        HttpGet getFlightRequest = new HttpGet(MessageFormat.format(

```

```

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
", applicationId,
    flightId));
getFlightRequest.addHeader("Authorization", "Bearer " +
accessToken);
getFlightRequest.addHeader("User-Agent", "Java");
JsonObject getFlightResponse =
httpClient.execute(getFlightRequest, responseHandler);

// Delete existing in-progress submission
JsonObject submissionToRemove =
getFlightResponse.getJsonObject("pendingFlightSubmission");
if(submissionToRemove != null){
    String submissionToRemoveId =
submissionToRemove.getString("id");
    HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
        applicationId, flightId, submissionToRemoveId));
    deleteSubmissionRequest.addHeader("Authorization", "Bearer "
+ accessToken);
    deleteSubmissionRequest.addHeader("User-Agent", "Java");
    httpClient.execute(deleteSubmissionRequest,
responseHandler);
}

// Create submission
HttpPost createSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions",
        applicationId, flightId));
    createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
    createSubmissionRequest.addHeader("User-Agent", "Java");
    JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);
    String submissionId = createSubmissionResponse.getString("id");
    String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

// Update submission
HttpPut updateSubmissionRequest = new
HttpPut(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
        applicationId, flightId, submissionId));
    updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
    updateSubmissionRequest.addHeader("Content-type",
"application/json");

```

```

        updateSubmissionRequest.addHeader("User-Agent", "Java");
        updateSubmissionRequest.setEntity(new
StringEntity(flightSubmissionRequestJson));
        JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

        // Upload images and packages in a zip file
        UploadZipFile(fileUploadUrl, zipFilePath);

        // Commit submission
        HttpPost commitSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}/commit",
        applicationId, flightId, submissionId));
        commitSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        commitSubmissionRequest.addHeader("User-Agent", "Java");
        JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);

        System.out.println(commitSubmissionResponse.getString("status"));

        // Pull submission status until commit process is completed
        HttpGet getSubmissionStatusRequest = new
HttpGet(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/applications/{0}/flights/{1}
/submissions/{2}",
        applicationId, flightId, submissionId));
        getSubmissionStatusRequest.addHeader("Authorization", "Bearer "
+ accessToken);
        getSubmissionStatusRequest.addHeader("User-Agent", "Java");
        JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
        responseHandler);
        String submissionStatus =
getSubmissionStatusResponse.getString("status");
        while(submissionStatus.equals("CommitStarted")){
            Thread.sleep(60*1000);
            getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
            submissionStatus =
getSubmissionStatusResponse.getString("status");
        }
        System.out.println(submissionStatus);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {

```

```

        httpclient.close();
    }
}

public static void CreateNewInAppProduct(String accessToken, String
inAppProductRequestJson)
    throws InterruptedException, IOException {

    CloseableHttpClient httpclient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Create a new in-app-product
        HttpPost createInAppProductRequest = new HttpPost(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts");
        createInAppProductRequest.addHeader("Authorization", "Bearer " +
accessToken);
        createInAppProductRequest.addHeader("Content-type",
"application/json");
        createInAppProductRequest.addHeader("User-Agent", "Java");
        createInAppProductRequest.setEntity(new
StringEntity(inAppProductRequestJson));
        JsonObject createInAppProductResponse =
httpclient.execute(createInAppProductRequest,
            responseHandler);
        String inAppProductId =
createInAppProductResponse.getString("id");
        System.out.println(createInAppProductResponse.toString());

        // Delete created in-app-product
        HttpDelete deleteInAppProductRequest = new HttpDelete(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/" +
inAppProductId);
        deleteInAppProductRequest.addHeader("Authorization", "Bearer " +
accessToken);
        deleteInAppProductRequest.addHeader("User-Agent", "Java");
        httpclient.execute(deleteInAppProductRequest, responseHandler);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpclient.close();
    }
}

public static void SubmitNewInAppProductSubmission(String accessToken,
String inAppProductId,
    String iapSubmissionRequestJson, String zipFilePath)

```

```

throws InterruptedException, IOException {

    CloseableHttpClient httpClient = HttpClients.createDefault();
    ResponseHandler<JsonObject> responseHandler =
CreateJsonResponseHandler();

    try {
        // Get the in-app-product
        HttpGet getInAppProductRequest = new HttpGet(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/" +
inAppProductId);
        getInAppProductRequest.addHeader("Authorization", "Bearer " +
accessToken);
        getInAppProductRequest.addHeader("User-Agent", "Java");
        JsonObject getInAppProductResponse =
httpClient.execute(getInAppProductRequest, responseHandler);

        // Delete existing in-progress submission
        JsonObject submissionToRemove =
getInAppProductResponse.getJSONObject(
            "pendingInAppProductSubmission");
        if(submissionToRemove != null){
            String submissionToRemoveId =
submissionToRemove.getString("id");
            HttpDelete deleteSubmissionRequest = new
HttpDelete(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}",

                inAppProductId, submissionToRemoveId));
            deleteSubmissionRequest.addHeader("Authorization", "Bearer "
+ accessToken);
            deleteSubmissionRequest.addHeader("User-Agent", "Java");
            httpClient.execute(deleteSubmissionRequest,
responseHandler);
        }

        // Create submission
        HttpPost createSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s",

                inAppProductId));
            createSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
            createSubmissionRequest.addHeader("User-Agent", "Java");
            JsonObject createSubmissionResponse =
httpClient.execute(createSubmissionRequest, responseHandler);
            String submissionId = createSubmissionResponse.getString("id");
            String fileUploadUrl =
createSubmissionResponse.getString("fileUploadUrl");

        // Update submission

```



```

        HttpPut updateSubmissionRequest = new
HttpPut(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}",

        inAppProductId, submissionId));
        updateSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        updateSubmissionRequest.addHeader("Content-type",
"application/json");
        updateSubmissionRequest.setEntity(new
StringEntity(iapSubmissionRequestJson));
        JsonObject updateSubmissionResponse =
httpClient.execute(updateSubmissionRequest, responseHandler);

        // Upload images and packages in a zip file
        UploadZipFile(fileUploadUrl, zipFilePath);

        // Commit submission
        HttpPost commitSubmissionRequest = new
HttpPost(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}/commit",

        inAppProductId, submissionId));
        commitSubmissionRequest.addHeader("Authorization", "Bearer " +
accessToken);
        commitSubmissionRequest.addHeader("User-Agent", "Java");
        JsonObject commitSubmissionResponse =
httpClient.execute(commitSubmissionRequest, responseHandler);

        System.out.println(commitSubmissionResponse.getString("status"));

        // Pull submission status until commit process is completed
        HttpGet getSubmissionStatusRequest = new
HttpGet(MessageFormat.format(

"https://manage.devcenter.microsoft.com/v1.0/my/inappproducts/{0}/submission
s/{1}/status",

        inAppProductId, submissionId));
        getSubmissionStatusRequest.addHeader("Authorization", "Bearer "
+ accessToken);
        getSubmissionStatusRequest.addHeader("User-Agent", "Java");
        JsonObject getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest,
responseHandler);
        String submissionStatus =
getSubmissionStatusResponse.getString("status");
        while(submissionStatus.equals("CommitStarted")){
            Thread.sleep(60*1000);
            getSubmissionStatusResponse =
httpClient.execute(getSubmissionStatusRequest, responseHandler);
            submissionStatus =
getSubmissionStatusResponse.getString("status");
        }

```

```

        System.out.println(submissionStatus);

    } catch (HttpResponseException e){
        e.printStackTrace();
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpClient.close();
    }
}

private static void UploadZipFile(String fileUploadUrl, String
zipFilePath) throws IOException{
    CloseableHttpClient httpClient = HttpClients.createDefault();
    File uploadFile = new File(zipFilePath);

    // Encode '+', otherwise it will be decoded as ' '
    HttpPut uploadFileRequest = new HttpPut(fileUploadUrl.replace("+",
"%2B"));
    uploadFileRequest.addHeader("x-ms-blob-type", "BlockBlob");
    uploadFileRequest.setEntity(new FileEntity(uploadFile));

    CloseableHttpResponse uploadFileResponse;
    try {
        uploadFileResponse = httpClient.execute(uploadFileRequest);
        System.out.println(uploadFileResponse.getStatusLine());
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        httpClient.close();
    }
}

private static ResponseHandler<JsonObject> CreateJsonResponseHandler(){
    ResponseHandler<JsonObject> responseHandler = new
ResponseHandler<JsonObject>() {
        @Override
        public JsonObject handleResponse(HttpResponse response) throws
IOException {
            StatusLine statusLine = response.getStatusLine();
            HttpEntity entity = response.getEntity();
            JsonObject returnValue = null;
            if(entity != null){
                JsonReader reader =
Json.createReader(entity.getContent());
                returnValue = reader.readObject();
            }

            // Log response status and correlation ID
            System.out.println(statusLine);
            System.out.println(response.getFirstHeader("MS-

```

```

CorrelationId"));

        // Underlying failure reason is in response entity
        if (statusLine.getStatusCode() >= 300) {
            if(entity == null){
                throw new HttpResponseMessage(
                    statusLine.getStatusCode(),
                    statusLine.getReasonPhrase());
            }
            else{
                throw new HttpResponseMessage(
                    statusLine.getStatusCode(),
                    statusLine.getReasonPhrase() + " " +
returnValue.toString() );
            }
        }

        return returnValue;
    }
};

return responseHandler;
}
}

```

Related topics

- [Create and manage submissions using Microsoft Store services](#)

Java sample: app submission with game options and trailers

Article • 01/04/2023

This article provides Java code examples that demonstrate how to use the [Microsoft Store submission API](#) for these tasks:

- Obtain an Azure AD access token to use with the Microsoft Store submission API.
- Create an app submission
- Configure Store listing data for the app submission, including the [gaming](#) and [trailers](#) advanced listing options.
- Upload the ZIP file containing the packages, listing images, and trailer files for the app submission.
- Commit the app submission.

Create an app submission

The `CreateAndSubmitSubmissionExample` class implements a `main` program that calls other example methods to use the Microsoft Store submission API to create and commit an app submission that contains game options and a trailer. To adapt this code for your own use:

- Assign the `tenantId` variable to the tenant ID for your app, and assign the `clientId` and `clientSecret` variables to the client ID and key for your app. For more information, see [How to associate an Azure AD application with your Partner Center account](#)
- Assign the `applicationId` variable to the [Store ID](#) of the app for which you want to create a submission.

Java

```
import java.io.IOException;
import java.text.MessageFormat;
import javax.json.Json;
import javax.json.JsonArrayBuilder;
import javax.json.JsonObject;
import javax.json.JsonObjectBuilder;
import org.apache.http.client.ClientProtocolException;

public class CreateAndSubmitSubmissionExample {

    public static void main(String[] args) {
```

```

        // Add your tenant ID, client ID, and client secret here.
        String tenantId = "";
        String clientId = "";
        String clientSecret = "";
        DevCenterAccessTokenClient accessTokenClient = new
DevCenterAccessTokenClient(tenantId, clientId, clientSecret);
        String accessToken =
accessTokenClient.getAccessToken("https://manage.devcenter.microsoft.com");

        // The application ID is taken from your app dashboard page's URI in
Dev Center,
        // e.g. https://developer.microsoft.com/en-
us/dashboard/apps/{application_id}/
        String applicationId = "";
        DevCenterClient devCenter = new
DevCenterClient("https://manage.devcenter.microsoft.com", accessToken);

        try {
            // Get the application object, and cancel any in progress
submissions.
            JsonObject app =
devCenter.getApplicationJsonObject(applicationId);
            JsonObject inProgressSubmission =
app.getJsonObject("pendingApplicationSubmission");
            if (inProgressSubmission != null) {
                String inProgressSubmissionId =
inProgressSubmission.getString("id");
                devCenter.cancelInProgressSubmission(applicationId,
inProgressSubmissionId);
            }

            // Create a new submission, based on the last published
submission.
            JsonObject submission =
devCenter.createSubmission(applicationId);
            String submissionId = submission.getString("id");

            // JsonObjects are immutable, so we'll build up our changes to
the submission and
            // then merge it with the submission object.
            JsonObjectBuilder submissionChanges =
Json.createObjectBuilder();

            // The following fields are required:
            submissionChanges.add("applicationCategory", "Games_Fighting");
            submissionChanges.add("listings", getListingsObject());
            submissionChanges.add("pricing", getPricingObject());
            submissionChanges.add("packages",
Json.createArrayBuilder().add(getPackageObject()));
            submissionChanges.add("allowTargetFutureDeviceFamilies",
getDeviceFamiliesObject());

            // Add new Gaming Options to the submission.
            JsonObject gamingOptions = getGamingOptionsJsonObject();

```

```

        submissionChanges.add("gamingOptions",
Json.createArrayBuilder().add(gamingOptions));

        // Add new Trailers to the submission.
        JsonObject trailer = getTrailerObject();
        submissionChanges.add("trailers",
Json.createArrayBuilder().add(trailer));

        // Continue updating the submission_json object with additional
options as needed.
        // After you've finished, call the Update API with the code
below to save it:
        JsonObject submissionToUpdate = mergeJsonObjects(submission,
submissionChanges.build());
        JsonObject updatedSubmission =
devCenter.updateSubmission(applicationId, submissionId, submissionToUpdate);

        // All images and packages should be located in a single ZIP
file. In the submission JSON,
        // the file names for all objects requiring them (icons,
packages, etc.) must exactly
        // match the file names from the ZIP file.
        String zipFilePath = "";
        devCenter.uploadZipFileForSubmission(applicationId,
submissionId, zipFilePath);

        // Committing the submission will start the submission process
for it. Once committed,
        // the submission can no longer be changed.
        devCenter.commitSubmission(applicationId, submissionId);

        // After committing, you can poll the commit API for the status
of the submission's process using
        // the following code.
        boolean waitingForCommitToStart = true;
        while (waitingForCommitToStart) {
            String status = devCenter.getSubmissionStatus(applicationId,
submissionId);
            System.out.println(MessageFormat.format("Submission status:
{0}", status));
            waitingForCommitToStart = status.equals("CommitStarted");
            if (waitingForCommitToStart) {
                try {
                    Thread.sleep(60000); // Wait one minute to check Dev
Center again.
                } catch (InterruptedException iex) {
                    System.out.println("The sleep was interrupted.
Checking Dev Center now.");
                }
            }
        }
    } catch (ClientProtocolException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

```

```

    }
}

private static JsonObject mergeJsonObjects(JsonObject baseObject,
JsonObject mergeObject) {
    JsonObjectBuilder builder = Json.createObjectBuilder();

    for (String baseKey : baseObject.keySet()) {
        if (mergeObject.containsKey(baseKey)) {
            builder.add(baseKey, mergeObject.get(baseKey));
        } else {
            builder.add(baseKey, baseObject.get(baseKey));
        }
    }

    for (String mergeKey : mergeObject.keySet()) {
        if (!baseObject.containsKey(mergeKey)) {
            builder.add(mergeKey, mergeObject.get(mergeKey));
        }
    }

    return builder.build();
}

private static JsonObject getListingsObject() {
    // This structure holds basic information to display in the store.
    JsonObjectBuilder baseListing = Json.createObjectBuilder();
    baseListing.add("copyrightAndTrademarkInfo", "(C) 2017 Microsoft");
    baseListing.add("licenseTerms",
"http://example.com/licenseTerms.aspx");
    baseListing.add("privacyPolicy",
"http://example.com/privacyPolicy.aspx");
    baseListing.add("supportContact", "support@example.com");
    baseListing.add("websiteUrl", "http://example.com");
    baseListing.add("description", "A sample game showing off gameplay
options code.");
    baseListing.add("releaseNotes", "Initial release");

    // The title of the app must match a reserved name for the app in
Dev Center.
    // If it doesn't, attempting to update the submission will fail.
    baseListing.add("title", "Super Dev Center API Simulator 2017");

    // Up to 7 keywords may be provided in a listing.
    JsonArrayBuilder keywords = Json.createArrayBuilder();

    keywords.add("SampleApp").add("SampleFightingGame").add("GameOptions");
    baseListing.add("keywords", keywords);

    JsonArrayBuilder features = Json.createArrayBuilder();
    features.add("Doesn't crash");
    features.add("Likes to eat chips");
    baseListing.add("features", features);

    // If your app works better with specific hardware (or needs it),

```

you can

```
// add or update values here.
JsonArrayBuilder hardwarePreferences = Json.createArrayBuilder();
hardwarePreferences.add("Keyboard");
hardwarePreferences.add("Mouse");
baseListing.add("hardwarePreferences", hardwarePreferences);

JsonArrayBuilder images = Json.createArrayBuilder();

// There are several types of images available; at least one
screenshot
// is required.
JsonObjectBuilder image = Json.createObjectBuilder();
image.add("fileName", "tile.png");
image.add("description", "The tile as it appears in the store.");
image.add("imageType", "Icon");
images.add(image);
baseListing.add("images", images);

JsonObjectBuilder listing = Json.createObjectBuilder();
listing.add("baseListing", baseListing);

// If there are any specific overrides to above information for
Windows 8,
// Windows 8.1, Windows Phone 7.1, 8.0, or 8.1, you can add
information here.
listing.add("platformOverrides", Json.createObjectBuilder());

JsonObjectBuilder listings = Json.createObjectBuilder();
// Each listing is targeted at a specific language-locale code, e.g.
EN-US.
listings.add("en-us", listing);
return listings.build();
}

private static JsonObject getPackageObject() {
    JsonObjectBuilder pkg = Json.createObjectBuilder();
    // The file name is relative to the root of the uploaded ZIP file.
    pkg.add("fileName", "bin/super_dev_ctr_api_sim.appxupload");

    // If you haven't begun to upload the file yet, set this value to
    "PendingUpload".
    pkg.add("fileStatus", "PendingUpload");
    return pkg.build();
}

private static JsonObject getPricingObject() {
    JsonObjectBuilder pricing = Json.createObjectBuilder();

    // How long the trial period is, if one is allowed. Valid values are
    NoFreeTrial,
    // OneDay, SevenDays, FifteenDays, ThirtyDays, or TrialNeverExpires.
    pricing.add("trialPeriod", "NoFreeTrial");

    // Maps to the default price for the app.
```



```

        pricing.add("priceId", "Free");

        // If you'd like to offer your app in different markets at different
prices, you
        // can provide priceId values per language/locale code.
        pricing.add("marketSpecificPricing",
Json.createObjectBuilder().build());
        return pricing.build();
    }

    private static JsonObject getDeviceFamiliesObject() {
        JsonObjectBuilder futureDeviceFamilies = Json.createObjectBuilder();

        // Supported values are Desktop, Mobile, Xbox, and Holographic. To
make
        // the app available on that specific platform, set the value to
True.
        futureDeviceFamilies.add("Desktop", true);
        futureDeviceFamilies.add("Mobile", false);
        futureDeviceFamilies.add("Xbox", true);
        futureDeviceFamilies.add("Holographic", false);
        return futureDeviceFamilies.build();
    }

    private static JsonObject getGamingOptionsJsonObject() {
        JsonObjectBuilder gamingOptions = Json.createObjectBuilder();

        // The genres of your game.
        JsonArrayBuilder genres = Json.createArrayBuilder();
        genres.add("Games_Fighting");
        gamingOptions.add("genres", genres);

        // Set this to true if your game supports local multiplayer. This
field
        // is required.
        gamingOptions.add("isLocalMultiplayer", true);

        // If local multiplayer is supported, you must provide the minimum
and
        // maximum players supported. Valid values are between 2 and 1000
inclusive.
        gamingOptions.add("localMultiplayerMinPlayers", 2);
        gamingOptions.add("localMultiplayerMaxPlayers", 4);

        // Set this to True if your game supports local co-op play. This
field is required.
        gamingOptions.add("isLocalCooperative", true);

        // If local co-op is supported, you must provide the minimum and
maximum players
        // supported. Valid values are between 2 and 1000 inclusive.
        gamingOptions.add("localCooperativeMinPlayers", 2);
        gamingOptions.add("localCooperativeMaxPlayers", 4);
    }

```

```

        // Set this to True if your game supports online multiplayer. This
        field is required.
        gamingOptions.add("isOnlineMultiplayer", true);

        // If online multiplayer is supported, you must provide the minimum
        and maximum players
        // supported. Valid values are between 2 and 1000 inclusive.
        gamingOptions.add("onlineMultiplayerMinPlayers", 2);
        gamingOptions.add("onlineMultiplayerMaxPlayers", 4);

        // Set this to true if your game supports online co-op play. This
        field is required.
        gamingOptions.add("isOnlineCooperative", true);

        // If online co-op is supported, you must provide the minimum and
        maximum players
        // supported. Valid values are between 2 and 1000 inclusive.
        gamingOptions.add("onlineCooperativeMinPlayers", 2);
        gamingOptions.add("onlineCooperativeMaxPlayers", 4);

        // If your game supports broadcasting a stream to other players, set
        this field to True.
        // This field is required.
        gamingOptions.add("isBroadcastingPrivilegeGranted", true);

        // If your game supports cross-device play (e.g. a player can play
        on an Xbox One with
        // their friend who's playing on a PC), set this field to True. This
        field is required.
        gamingOptions.add("isCrossPlayEnabled", true);

        // If your game supports Kinect usage, set this field to "Enabled",
        otherwise, set it to
        // "Disabled". This field is required.
        gamingOptions.add("kinectDataForExternal", "Disabled");

        // Free text about any other peripherals that your game supports.
        This field is optional.
        gamingOptions.add("otherPeripherals", "Supports the usage of all
        fighting joysticks.");

        return gamingOptions.build();
    }

    private static JsonObject getTrailerObject() {
        JsonObjectBuilder trailer = Json.createObjectBuilder();

        // This is the filename of the trailer. The file name is a relative
        path to the
        // root of the ZIP file to be uploaded to the API.
        trailer.add("VideoFileName",
        "trailers/main/my_awesome_trailer.mpeg");

        // Aside from the video itself, a trailer can have metadata assets
        including a title and images

```

```

        // such as screenshots or alternate images. These are keyed by
        market code (see the end of this
        // method for an example.
        JsonObjectBuilder trailerAssetsByCountry =
        Json.createObjectBuilder();

        JsonObjectBuilder trailerAssetSet = Json.createObjectBuilder();

        // The title of the trailer to display in the store.
        trailerAssetSet.add("Title", "Main Trailer");

        // The list of images provided with the trailer that are shown when
        the trailer isn't playing.
        JsonArrayBuilder trailerImageAssets = Json.createArrayBuilder();

        JsonObjectBuilder mainTrailerImage = Json.createObjectBuilder();

        // The file name of the image. The file name is a relative path to
        the root of the ZIP
        // file to be uploaded to the API.
        mainTrailerImage.add("FileName", "trailers/main/thumbnail.png");

        // A plaintext description of what the image represents.
        mainTrailerImage.add("Description", "The thumbnail for the trailer
        shown before the user clicks play");
        trailerImageAssets.add(mainTrailerImage);

        // Add a second image.
        JsonObjectBuilder altImage = Json.createObjectBuilder();
        altImage.add("FileName", "trailers/main/alt-img.png");
        altImage.add("Description", "The image to show after the trailer
        plays");
        trailerImageAssets.add(altImage);

        trailerAssetSet.add("ImageList", trailerImageAssets);

        // This line creates the trailer asset for en-us.
        trailerAssetsByCountry.add("en-us", trailerAssetSet);

        trailer.add("TrailerAssets", trailerAssetsByCountry);

        return trailer.build();
    }
}

```

Obtain an Azure AD access token

The `DevCenterAccessTokenClient` class defines a helper method that uses the your `tenantId`, `clientId` and `clientSecret` values to create an Azure AD access token to use with the Microsoft Store submission API.

```

import java.io.ByteArrayInputStream;
import java.io.IOException;
import java.io.UnsupportedEncodingException;
import java.text.MessageFormat;
import javax.json.Json;
import javax.json.JsonReader;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.ResponseHandler;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.entity.StringEntity;
import org.apache.http.impl.client.BasicResponseHandler;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;

/**
 * A client for getting access tokens to the Dev Center API.
 * @author Microsoft
 */
public final class DevCenterAccessTokenClient {
    private String tenantId;
    private String clientId;
    private String clientSecret;

    /**
     * Creates a new access token client for Dev Center.
     * @param tenantId Your tenant ID for the app.
     * @param clientId Your client ID for the app.
     * @param clientSecret Your client secret string for the app.
     */
    public DevCenterAccessTokenClient(String tenantId, String clientId,
String clientSecret) {
        this.tenantId = tenantId;
        this.clientId = clientId;
        this.clientSecret = clientSecret;
    }

    /**
     * Gets an access token for the specific resource.
     * @param resource The full URI to the resource to be accessed.
     * @return An access token for that resource, good for one hour.
     */
    public String getAccessToken(String resource) {
        // Generate access token. Access token is valid for 1 hour.
        String tokenEndpoint =
"https://login.microsoftonline.com/{0}/oauth2/token";

        HttpPost tokenRequest = new
HttpPost(MessageFormat.format(tokenEndpoint, this.tenantId));
        String tokenRequestBody =
MessageFormat.format("grant_type=client_credentials&client_id=
{0}&client_secret={1}&resource={2}", this.clientId, this.clientSecret,
resource);

```

```

        tokenRequest.setEntity(new StringEntity(tokenRequestBody, "utf-8"));
        tokenRequest.addHeader("Content-Type", "application/x-www-form-
urlencoded; charset=utf-8");

        CloseableHttpClient httpclient = HttpClients.createDefault();
        ResponseHandler<String> responseHandler = new
BasicResponseHandler();

        try {
            String tokenResponse = httpclient.execute(tokenRequest,
responseHandler);
            JsonReader reader = Json.createReader(new
ByteArrayInputStream(tokenResponse.getBytes("UTF-8")));
            String accessToken =
reader.readObject().getString("access_token");

            return accessToken;
        } catch (UnsupportedEncodingException e) {
            e.printStackTrace();
        } catch (ClientProtocolException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }

        return null;
    }
}

```

Helper methods to invoke the submission API and upload submission files

The `DevCenterClient` class defines helper methods that invoke a variety of methods in the Microsoft Store submission API and upload the ZIP file containing the packages, listing images, and trailer files for the app submission.

Java

```

import java.io.File;
import java.io.IOException;
import java.io.StringWriter;
import java.text.MessageFormat;
import javax.json.Json;
import javax.json.JsonObject;
import javax.json.JsonReader;
import org.apache.http.HttpEntity;
import org.apache.http.HttpResponse;
import org.apache.http.StatusLine;
import org.apache.http.client.ClientProtocolException;
import org.apache.http.client.HttpResponseException;

```

```

import org.apache.http.client.ResponseHandler;
import org.apache.http.client.methods.HttpDelete;
import org.apache.http.client.methods.HttpGet;
import org.apache.http.client.methods.HttpPost;
import org.apache.http.client.methods.HttpPut;
import org.apache.http.client.methods.HttpUriRequest;
import org.apache.http.entity.FileEntity;
import org.apache.http.entity.StringEntity;
import org.apache.http.impl.client.CloseableHttpClient;
import org.apache.http.impl.client.HttpClients;

/**
 * A client for accessing commands in the Dev Center API.
 * @author Microsoft
 */
public final class DevCenterClient implements ResponseHandler<JsonObject> {
    private String baseUri;
    private String accessToken;

    /**
     * Creates a new dev center client instance.
     * @param baseUri The base URI for the Dev Center API.
     * @param accessToken The access token for web call authentication.
     */
    public DevCenterClient(String baseUri, String accessToken) {
        this.baseUri = baseUri;
        this.accessToken = accessToken;
    }

    /**
     * Returns the application JSON object from the Dev Center API.
     * @param applicationId The application ID.
     * @return A JSON object from Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */
    public JsonObject getApplicationJsonObject(String applicationId) throws
ClientProtocolException, IOException {
        String path = MessageFormat.format("/v1.0/my/applications/{0}",
applicationId);
        return get(path);
    }

    /**
     * Cancels and deletes the in-progress submission from the application.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @return A JSON object from Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */

```

```

    public JsonObject cancelInProgressSubmission(String applicationId,
String submissionId) throws ClientProtocolException, IOException {
        String path =
MessageFormat.format("/v1.0/my/applications/{0}/submissions/{1}",
applicationId, submissionId);
        return delete(path);
    }

    /**
     * Creates a new submission in Dev Center.
     * @param applicationId The application ID.
     * @return The submission JSON object from Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */
    public JsonObject createSubmission(String applicationId) throws
ClientProtocolException, IOException {
        String path =
MessageFormat.format("/v1.0/my/applications/{0}/submissions",
applicationId);
        return post(path);
    }

    /**
     * Updates the submission in Dev Center.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @param submission The submission JSON object.
     * @return The updated submission JSON object from Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */
    public JsonObject updateSubmission(String applicationId, String
submissionId, JsonObject submission) throws ClientProtocolException,
IOException {
        String path =
MessageFormat.format("/v1.0/my/applications/{0}/submissions/{1}",
applicationId, submissionId);
        return put(path, submission);
    }

    /**
     * Gets the submission in Dev Center.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @return The submission JSON object from Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */

```

```

    public JsonObject getSubmission(String applicationId, String
submissionId) throws ClientProtocolException, IOException {
        String path =
MessageFormat.format("/v1.0/my/applications/{0}/submissions/{1}",
applicationId, submissionId);
        return get(path);
    }

    /**
     * Commits the submission to Dev Center.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */
    public void commitSubmission(String applicationId, String submissionId)
throws ClientProtocolException, IOException {
        String path =
MessageFormat.format("/v1.0/my/applications/{0}/submissions/{1}/commit",
applicationId, submissionId);
        post(path);
    }

    /**
     * Returns the submission status of this submission.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @return The status of the submission in Dev Center.
     * @throws IOException Thrown when a serialization exception occurs on
the read.
     * @throws ClientProtocolException Thrown when an HTTP communication
error occurs.
     */
    public String getSubmissionStatus(String applicationId, String
submissionId) throws ClientProtocolException, IOException {
        JsonObject response = getSubmission(applicationId, submissionId);
        String status = response.getString("status");
        if (status == null || status.isEmpty())
        {
            return "Unknown";
        }

        return status;
    }

    /**
     * Uploads a ZIP archive containing the binaries, image assets,
trailers, and other components to the submission.
     * @param applicationId The application ID.
     * @param submissionId The submission ID.
     * @param zipFilePath The file path to the ZIP file.
     * @throws IOException Thrown when a serialization exception occurs on
the read.

```



```

    * @throws ClientProtocolException Thrown when an HTTP communication
    error occurs.
    */
    public void uploadZipFileForSubmission(String applicationId, String
    submissionId, String zipFilePath) throws ClientProtocolException,
    IOException {
        JsonObject submission = getSubmission(applicationId, submissionId);
        String fileUploadUrl = submission.getString("fileUploadUri");

        CloseableHttpClient httpclient = HttpClients.createDefault();
        File uploadFile = new File(zipFilePath);
        HttpPut uploadFileRequest = new HttpPut(fileUploadUrl.replace("+",
"%2B")); // Encode '+', otherwise it will be decoded as ' '
        uploadFileRequest.addHeader("x-ms-blob-type", "BlockBlob");
        uploadFileRequest.setEntity(new FileEntity(uploadFile));
        httpclient.execute(uploadFileRequest);
    }

    private JsonObject get(String path) throws ClientProtocolException,
    IOException {
        HttpGet request = new HttpGet(this.baseUri + path);
        return invoke(request);
    }

    private JsonObject put(String path, JsonObject obj) throws
    ClientProtocolException, IOException {
        HttpPut request = new HttpPut(this.baseUri + path);
        request.addHeader("Content-Type", "application/json; charset=utf-
8");
        request.setEntity(new StringEntity(SerializeJsonObject(obj)));
        return invoke(request);
    }

    private JsonObject post(String path) throws ClientProtocolException,
    IOException {
        return post(path, null);
    }

    private JsonObject post(String path, JsonObject obj) throws
    ClientProtocolException, IOException {
        HttpPost request = new HttpPost(this.baseUri + path);
        request.addHeader("Content-Type", "application/json; charset=utf-
8");
        if (obj != null)
        {
            request.setEntity(new StringEntity(SerializeJsonObject(obj)));
        }
        return invoke(request);
    }

    private JsonObject delete(String path) throws ClientProtocolException,
    IOException {
        HttpDelete request = new HttpDelete(this.baseUri + path);
        return invoke(request);
    }

```

```

    private JsonObject invoke(HttpUriRequest request) throws
ClientProtocolException, IOException {
        CloseableHttpClient client = HttpClients.createDefault();
        request.addHeader("Authorization", "Bearer " + accessToken);
        request.addHeader("User-Agent", "Java");
        JsonObject response = client.execute(request, this);
        return response;
    }

    public JsonObject handleResponse(HttpResponse response) throws
ClientProtocolException, IOException {
        StatusLine status = response.getStatusLine();
        int statusCode = status.getStatusCode();
        String reasonPhrase = status.getReasonPhrase();
        HttpEntity entity = response.getEntity();
        JsonObject returnValue = null;
        if(entity != null && entity.getContentLength() != 0) {
            JsonReader reader = Json.createReader(entity.getContent());
            returnValue = reader.readObject();
        }

        if (statusCode < 200 || statusCode > 299) {
            if (returnValue != null) {
                throw new HttpResponseException(statusCode,
returnValue.toString());
            }

            throw new HttpResponseException(statusCode, reasonPhrase);
        }

        return returnValue;
    }

    private static String SerializeJsonObject(JsonObject obj) {
        StringWriter writer = new StringWriter();
        Json.createWriter(writer).writeObject(obj);
        String body = writer.toString();
        return body;
    }
}

```

Related topics

- [Create and manage submissions using Microsoft Store services](#)

Python sample: submissions for apps, add-ons, and flights

Article • 01/04/2023

This article provides Python code examples that demonstrate how to use the [Microsoft Store submission API](#) for these tasks:

- [Obtain an Azure AD access token](#)
- [Create an add-on](#)
- [Create a package flight](#)
- [Create an app submission](#)
- [Create an add-on submission](#)
- [Create a package flight submission](#)

Obtain an Azure AD access token

The following example demonstrates how to [obtain an Azure AD access token](#) that you can use to call methods in the Microsoft Store submission API. After you obtain a token, you have 60 minutes to use this token in calls to the Microsoft Store submission API before the token expires. After the token expires, you can generate a new token..

Python

```
import http.client, json

tenantId = "" # Your tenant ID
clientId = "" # Your client ID
clientSecret = "" # Your client secret

tokenEndpoint = "https://login.microsoftonline.com/{0}/oauth2/token"
tokenResource = "https://manage.devcenter.microsoft.com"

tokenRequestBody = "grant_type=client_credentials&client_id={0}&client_secret={1}&resource={2}".format(clientId, clientSecret, tokenResource)
headers = {"Content-Type": "application/x-www-form-urlencoded; charset=utf-8"}
tokenConnection = http.client.HTTPSConnection("login.microsoftonline.com")
tokenConnection.request("POST", "{0}/oauth2/token".format(tenantId), tokenRequestBody, headers=headers)

tokenResponse = tokenConnection.getresponse()
print(tokenResponse.status)
tokenJson = json.loads(tokenResponse.read().decode())
print(tokenJson["access_token"])
```

```
tokenConnection.close()
```

Create an add-on

The following example demonstrates how to [create](#) and then [delete](#) an add-on.

Python

```
import http.client, json

accessToken = "" # Your access token
iapRequestJson = "" # Your in-app-product request JSON

headers = {"Authorization": "Bearer " + accessToken,
           "Content-type": "application/json",
           "User-Agent": "Python"}
ingestionConnection =
http.client.HTTPSConnection("manage.devcenter.microsoft.com")

# Create a new in-app-product
ingestionConnection.request("POST", "/v1.0/my/inappproducts",
iapRequestJson, headers)
createIapResponse = ingestionConnection.getresponse()
print(createIapResponse.status)
print(createIapResponse.reason)
print(createIapResponse.headers["MS-CorrelationId"]) # Log correlation ID

iapJsonObject = json.loads(createIapResponse.read().decode())
inAppProductId = iapJsonObject["id"]

# Delete created in-app-product
ingestionConnection.request("DELETE", "/v1.0/my/inappproducts/" +
inAppProductId, "", headers)
deleteIapResponse = ingestionConnection.getresponse()
print(deleteIapResponse.status)
print(deleteIapResponse.headers["MS-CorrelationId"]) # Log correlation ID

ingestionConnection.close()
```

Create a package flight

The following example demonstrates how to [create](#) and then [delete](#) a package flight.

Python

```
import http.client, json, requests, time
from azure.storage.blob import BlobClient
```

```

accessToken = "" # Your access token
applicationId = "" # Your application ID
flightRequestJson = "" # Your flight request JSON

headers = {"Authorization": "Bearer " + accessToken,
           "Content-type": "application/json",
           "User-Agent": "Python"}
ingestionConnection =
http.client.HTTPSConnection("manage.devcenter.microsoft.com")

# Create a new flight, a flight submission will be created together with the
flight

ingestionConnection.request("POST",
"/v1.0/my/applications/{0}/flights".format(applicationId),
flightRequestJson, headers)
createFlightResponse = ingestionConnection.getresponse()
print(createFlightResponse.status)
print(createFlightResponse.reason)
print(createFlightResponse.headers["MS-CorrelationId"]) # Log correlation
ID

flightJsonObject = json.loads(createFlightResponse.read().decode())
flightId = flightJsonObject["flightId"]
submissionId = flightJsonObject["pendingFlightSubmission"]["id"]

# Delete created flight
ingestionConnection.request("DELETE",
"/v1.0/my/applications/{0}/flights/{1}".format(applicationId, flightId), "",
headers)
deleteFlightResponse = ingestionConnection.getresponse()
print(deleteFlightResponse.status)
print(deleteFlightResponse.headers["MS-CorrelationId"]) # Log correlation
ID

ingestionConnection.close()

```

Create an app submission

The following example shows how to use several methods in the Microsoft Store submission API to create an app submission. To do this, the code creates a new submission as a clone of the last published submission, and then updates and commits the cloned submission to Partner Center. Specifically, the example performs these tasks:

1. To begin, the example [gets data for the specified app](#).
2. Next, it [deletes the pending submission for the app](#), if one exists.
3. It then [creates a new submission for the app](#) (the new submission is a copy of the last published submission).